

Utility companies exist across the United States, varying in size, the rebates and incentives they offer, and the eligibility requirements for obtaining them. The purpose of this memo is to explain how our team developed a Python-based pipeline to find, extract, and analyze utility companies' incentive information from their websites. This memo is meant to serve as an introduction and explanation of logic and code; for a more indepth explanation of set up and coding execution, refer to the README in the github repository

### **Backend Pipeline Explanations:**

The Python pipeline developed for this project consists of three main functions; the scraper, the analyzer (an initial agent and a filter), and a utility company discoverer. The purpose of this portion of the memo is to explain the logic behind the modules to help the users better understand how the pipeline functions.

#### **Scraper:**

The first portion of the pipeline was designed to gather all the relevant information found on a utility's website through a process called scraping. To do this, we used the Crawl4AI package, which is designed to crawl and scrape information from websites into structured data for further analysis by large language models (LLMs). To get a better understanding of Crawl4AI and our scraper, it is important to understand its core functions:

- **scrape\_single\_link:** `scrape_single_link` is the top-level function that orchestrates a full crawl. It takes a URL, a flag for whether to deep-crawl, and an optional truncation length. It sets up the crawl strategy, runs the crawler, processes each result, and saves everything to disk.
  - **The Two Crawl Modes** (both modes use the same scraping strategy)
    - **`use_deep_crawl = True`** — Builds a `BestFirstCrawlingStrategy` that follows linked subpages across the site, prioritizing pages that score highest for incentive-related keywords. It crawls up to 3 levels deep and stays within the seed domain via a `DomainFilter`. A `KeywordRelevanceScorer` and `SEOFilter` are layered on top to filter out low-signal pages before they are crawled.
    - **`use_deep_crawl = False`** — Skips all of the above and fetches only the single URL provided.
  - **Packages Used**
    - **Crawl4ai:** The core crawling library. The following components are used:
      - **AsyncWebCrawler** — The asynchronous browser-based crawler. Runs Chromium under the hood via Playwright.
      - **CrawlerRunConfig** — Bundles the strategy, cache mode, and other settings before passing to `arun()`.
      - **BestFirstCrawlingStrategy** — Deep-crawl strategy that scores and queues URLs by relevance rather than crawling everything blindly.

- **LXMLWebScrapingStrategy** — Uses the lxml parser to extract page content. Faster and more lenient than standard HTML parsers.
- **CacheMode.BYPASS** — Tells the crawler always to fetch fresh, ignoring any cached responses.
- **hashlib** — Generates a short 8-character MD5 hash of each page's URL, which becomes part of the output filename. This ensures unique filenames even when two pages share the same domain.
- **os** — Handles creating the scraped\_data/ output directory if it does not already exist, and building the file path for each saved .md file.
- **urllib.parse (urlparse)** — Extracts the netloc (domain) from each result's URL, used both to construct the filename and to enforce the same-domain check that prevents results from leaking onto external sites.
- **Output Files:** Each crawled page that returns success = True and a 200 status is written to scraped\_data/<domain>\_<hash>.md. Each file contains:
  - A --- SOURCE: <url> --- header at the top
  - The scraped markdown text of the page
  - Any auxiliary PDF or Excel content from process\_auxiliary\_files, appended under clearly marked section headers
  - If total text exceeds the truncation\_length (explained more in *Results from Different Models*), it is hard-cut with a truncation notice appended
- **process\_auxiliary\_files(html, base\_url)** is called on every crawled page to discover and extract embedded file content. It parses the page HTML for links, filters to same-domain only, and targets up to 3 PDFs and 2 Excel files. PDFs are ranked by \_score\_pdf\_url() before scraping, and Excel files are downloaded via aiohttp and converted to markdown tables using pandas.
- **\_scrape\_pdf\_with\_crawl4ai(pdf\_url)** handles PDF extraction using Crawl4AI's PDFContentScrapingStrategy inside a dedicated PDFCrawlerStrategy crawler. It attempts to extract fit\_markdown, falls back to raw\_markdown, and finally falls back to raw HTML if needed. It enforces a 120-second timeout per PDF.
- **\_score\_pdf\_url(url)** is a lightweight heuristic that scores a PDF URL by scanning it for grant-related keywords (worth +2 each) and bonus terms like "guide" or "form" (+1 each). This score is used to prioritize which PDFs are worth scraping when multiple are found on a page.

### Analyzer:

The scraped website information is then fed into a series of LLMs designed as analyzers, with the initial analyzer (llm\_agent.py) designed to find the program name, URL, concrete rebates, eligibility, and any

additional utility information. If there is no relevant information, it does not append the results to the analysis\_results folder. These results are then fed into another LLM designed to filter and act as a “final-stage quality control analyst”. The job of the filter is to discard any section that says "NOT RELEVANT", discard any conversational filler, and make sure to keep program details and eligibility requirements as bullet points under their respective headers, while also maintaining the initial output structure.

Both LLM’s take in similar parameters, only differing in the file path and prompt:

- **File Path** — the information the model takes in
  - Initial Analyzer (llm\_agent.py) — takes in the list of scraped .md (markdowns) files
  - Filter (processor.py) — takes in the results\_filepaths created by the initial analyzer
- **Provider** — which LLM backend to use. Passed into **build\_llm()** to determine how the model object is constructed. Swapping this lets you point either function at a different service without changing anything else
- **model** — The specific model to load within the chosen provider. Both functions default to the same model, but they can be given different ones based on goals. Different-sized models were tested (small, medium, and large), with further results in *Results from Different Models*
- **Temperature** — Controls how deterministic the LLM's output is, with 0 being most rigid and 1 being most creative. Both functions default to 0.1, which keeps responses consistent and structured, important here since the output needs to match specific headers (like ## PROGRAM DETAILS and ## ELIGIBILITY) to pass validation. A higher value would introduce more variation and increase the chance of malformed outputs being dropped.
- **Prompt** — what the LLM is designed to do
  - Initial Analyzer (TEMPLATE) — The LLM is acting as a rebate extraction tool. It reads a scraped page and decides whether it contains a real, active rebate or incentive program. If the page is just an article, blog post, or general energy advice, it's told to immediately abort with a NOT RELEVANT response, which the function code then catches and drops. If a real program is found, it extracts the program name, URL, rebate amounts, eligibility requirements, and utility company details into a strict markdown structure. The {source} and {document\_text} placeholders are filled in per file by chain.invoke().
  - Filter (FILTER\_TEMPLATE) — The LLM is acting as a cleanup pass. By this point, a domain's analysis file may contain a mix of valid extractions, leftover NOT RELEVANT strings that slipped through, empty fields, or malformed sections. The filter reads the entire accumulated file and rewrites it, keeping only the entries that have a complete, valid structure and discarding everything else. If nothing valid survives, it outputs NO REBATES FOUND. The {raw\_report} placeholder is filled with the full contents of the domain analysis file.

## URL Discoverer:

As an alternative to inputting known utility URLs, the URL discoverer was created to locate electric utilities that have not been captured in DSIRE by finding their official websites. This was accomplished with the use of OpenSERP, a search engine proxy that is executed locally and requires no APIs outside. The module asks OpenSERP for targeted search phrases and limits results to valid utility websites to then scrape and analyze. When running the pipeline, both `energy_search.py` and `app.py` must have the same local server address. For example, “<http://localhost:7070>.” (Make sure the local server addresses are all consistent in the settings.)

- **search\_topics:** A set of 20 search phrases created around the three major utility ownership types within the EIA-861 dataset, Cooperative, Municipal, and Investor Owned. Each phrase is joined with the U.S. state name when executing at runtime to compose a fully formed query (e.g., "electric cooperative rebate incentive program Texas"). This strategy scopes results both geographically and for a specific ownership type.
  - **is\_utility\_url:** A domain-level filter applied to each search result before a result is stored. Filters a result's domain against a blocklist of known aggregators, news operations, advocacy organizations, or non-utility government domains (e.g., DSIRE, EnergySage, NREL, Wikipedia, Bloomberg). Results on the blocklist are removed.
  - **load\_existing\_domains:** Reads the URL database at startup and builds a set of domains with www at the root. From there, during the search, if the domain obtained already exists (in the same base) in the set, it is then skipped (no saving to the output, so no duplicate). When results are searched for, any result whose domain is already present in this set will be skipped to prevent the same domain entry into the output.
  - **run\_state:** The top-level function that coordinates a discovery for one state. It loops over all search topics, queries OpenSERP, `is_utility_url`, and domain deduplication, outputs the passing on results to output Excel with: state, search query, url, page title, description, and discovery timestamp.
- **Packages Used**
  - **requests:** Sends HTTP GET requests to the local OpenSERP instance and retrieves JSON search results.
  - **Openpyxl:** Reads and writes Excel files for both the discovered URL output and the existing URL database.
  - **urllib.parse (urlparse):** Extracts and normalizes domains from URLs for filtering and deduplication.
  - **re:** Compiles the domain blocklist as a regular expression pattern for efficient matching against search result domains.

- **Output Files:** Any discovered URL passing the domain filter and deduplication will be written to `utility_urls_discovered.xlsx`. And each row will contain State, Search Query, URL, Page Title, Description, and Time Discovered.
  - **Merge module:** Following runs, the merge module collects the newly collected URLs into the master database (`Relevant_URLs.xlsx`). Domain-level deduplication, considering different pages on the same utility website, is applied to prevent redundant scraping in the downstream pipelines.
  - **load\_existing\_urls:** Reads the current database and builds both a full URL list and a bare-domain set used for deduplication.
  - **load\_discovered:** Reads `utility_urls_discovered.xlsx` and returns all rows where the URL field is populated.
  - **extract\_domain:** Normalizes a URL to its bare domain (by removing scheme, path, and `www.` prefix), which is the deciding factor for whether a newly discovered URL is considered a duplicate.
  - **build\_merged\_workbook:** Builds output Excel workbook with two sheets. "All URLs" keeps the original database URLs alongside newly added ones, with a dated separator row indicating the new additions. "New URLs" contains the additions with complete metadata for manual checking.
  - The merged database is saved back to `Relevant_URLs.xlsx`, replacing the original. The previous version is automatically renamed to `Relevant_URLs_backup_<timestamp>.xlsx` before the new file is written.

### **Streamlit Usage:**

The project was developed to be run within Streamlit, which is an open-source Python framework that allows you to convert your code into a more user-friendly interface than a Python terminal. Our framework includes different modes based on the structure of the sites and/or stored data, as well as desired results:

- **Excel files (list of URLs)** — provide a `.xlsx` with a list of URLs. The column containing the links should be titled "URLs" and will be run through our pipeline
- **Scraped Markdown** — If you have pre-scraped information from a website, you can upload the file for it to be analyzed (bypasses the scraping portion)
- **Singular Link** — attach a link or website that you wish to crawl, scrape, and analyze
- **Search for an incentive** — You can choose which search engine, state, search topics, and results per search topic to look through to derive their URLs, with which you can plug back into the Excel file input mode.

Streamlit returns rebate and incentive information for the given URL in both an Excel file and a markdown file.

### **Results from Different Models:**

Three different models were applied to the same website to determine how model choice impacted the results. The website chosen for this test was <https://www.anaheim.net/936/Energy-Rebates-Programs>. This was chosen due to its high density of information and rebates to scrape, making it easy to see differences in model performance. The models were chosen primarily based on differences in size, truncation length, and expected reasoning capacity.

#### **Small Model (llama 3.1:8b)**

The small-model condition used Llama 3.1 8B through Ollama, which Ollama lists as an 8.03-billion-parameter model with a 4.9 GB local model size. Llama 3.1 also supports a 128K-token context window, allowing it to process relatively long scraped inputs despite its small parameter count. This model can be run on most laptops with 8-16GB of RAM, and requires truncation of the scraped data to stay at or below 8000 to make sure the LLM context isn't overwhelmed. This can be changed on the Streamlit app.

On the Anaheim test page, Llama 3.1 8B performed better than expected on the initial analyzer step for a small local model. Its output identified a broad set of relevant Anaheim Public Utilities programs and generally preserved the required structure, including program names, URLs, rebate amounts, eligibility requirements, and utility information. The model successfully extracted detailed entries such as Heating & Cooling Systems, A/C Tune-Up, Refrigerator Recycling, Community Solar Discount, EV charger rebates, Leak Repair, Turf Replacement, and Utility Discount programs. Its results were less expansive than GPT-5.4 Pro, suggesting that the context window may have been insufficient for subpages containing larger blocks of information. This suggests that Llama 3.1 8B was capable of handling structured incentive extraction when the scraped text was sufficiently clear, though it may have missed some less direct or harder-to-find programs compared with the larger model. When performing the filtering step, the model size was an apparent limitation. Incentives were listed, but the output format and details surrounding each incentive were lost.

#### **Medium Model (gemma-4-31b)**

This model has 31 billion parameters, an approximately 20 GB model footprint, and a 256K-token context window. This model requires a laptop or desktop with 16-32 GB of RAM, and a truncation length of 12000-16000 characters.

Compared with the small 8B model, Gemma 4 31B represents a substantial increase in model capacity and memory requirements. This larger scale was expected to improve the model's ability to extract rebate and incentive information from noisy scraped text, preserve program details, and follow the required output structure. Its larger context window also made it better suited for longer webpages, PDFs, or pages containing multiple programs.

On the Anaheim page, Gemma 4 31B produced a more conservative extraction than expected for a medium-sized model. It identified several real Anaheim Public Utilities programs, including Go Digital and Save, TreePower, Battery Storage, Back-Up Power, myPower Savings, Weatherization, Dusk-to-Dawn Lighting, and Residential Energy Efficiency Rebates. However, it missed many major rebate categories that were captured by the other models, including detailed heating and cooling rebates, EV charger programs, refrigerator recycling, pool equipment, utility discount programs, and several commercial or water-related incentives. Its output was relatively clean and did not over-expand into as many loosely related city programs, but this came at the cost of lower recall. Overall, Gemma 4 31B appeared to favor precision and relevance over completeness, making it useful for cleaner extraction but weaker for comprehensive incentive discovery. Additionally, filtering was not performed with this model due to errors encountered when running it through the UW supercomputer. This means that whether it would maintain the desired structure during filtering is unknown.

### Large Model (GPT 5.4 Pro)

The large-model condition used GPT-5.4 Pro, which served as the highest-capability model tested. Unlike Llama 3.1 8B and Gemma 4 31B, GPT-5.4 Pro is a proprietary model, so its exact parameter count and model footprint are not publicly disclosed. However, it has a substantially larger reported context window than the other models, with a context length of approximately 1.05 million tokens. This larger context capacity was expected to make GPT-5.4 Pro better suited for processing complex webpages containing many rebates, eligibility rules, application details, and noisy scraped text. Because of its larger context window and expected stronger reasoning and instruction-following ability, GPT-5.4 Pro was expected to distinguish relevant rebate programs from unrelated page content, preserve specific financial and eligibility details, and return outputs in the required structure. However, this increased capability comes with trade-offs, including a much higher expected cost compared with smaller open models.

GPT-5.4 Pro produced the broadest extraction on the Anaheim test page. It identified the largest number of candidate programs, including many pages related to residential, commercial, water, electric vehicle, battery storage, utility discount, and municipal assistance. This indicates that GPT-5.4 Pro had the strongest recall and was most effective at finding additional incentives across linked pages, PDFs, and related Anaheim Public Utilities content. However, the larger output also introduced more noise, including duplicate programs, overlapping document versions, generic landing-page summaries, and some lower-relevance city programs. With the additional filtering step, however, these duplicates were removed and the overall structure was preserved. Overall, this model produced the cleanest results with the most applicable incentives.

### **Recommendations:**

This pipeline has been developed through many forms, optimizing features to best retrieve incentive information. Through the different trials we have separated variables that can and should not change, depending on the variability in technological capabilities and personal needs. For all parameters for the scraper constants, model types, and directory names can be accessed and changed in **config.py**

### **Parameters that should not be changed:**

- Temperature — We chose a value of 0.1 since it allows for better semantic understanding from our tests but is still firmly grounded and rooted in drawing directly from the page. We recommend keeping it at the previously hardcoded 0.1 but believe our project is usable within the range of 0 - 0.3 to either be more strict or to have slightly more creativity
- Prompts — Both the initial and final filters prompt or template have been optimized to reduce hallucinations and the inclusion of irrelevant information. While we don't suggest that you change this prompt much, if the structure of the markdown output is not optimal for your usage, change the prompt under "OUTPUT FORMATTING" in the TEMPLATE to better serve your needs.
- max\_depth — the number of levels to crawl beyond the starting page. We have set this to a depth of 3, and strongly recommend you keep it that way. A scrape of 2 misses a large amount of important information, but anything past 3 takes far too long due to the exponential nature of website scrawling. The same logic applies to MAX\_RANKED\_PDFS, MAX\_EXCEL\_FILES, MAX\_EXCEL\_SIZE\_BYTES, and PDF\_SCRAPE\_TIMEOUT

#### **Parameters that allow changes:**

- Model type:
  - Smaller models tend to work relatively well for the initial analysis step, but struggle when filtering due to an insufficient context window. If smaller models are used, it is suggested that you refer to the unfiltered analysis results folder for further details that may be lost in filtering.
  - There are other, untested avenues and configurations that we believe would increase performance. First, a paid AI API would decrease the load on the local computer and could be hosted on a Streamlit community cloud for multiple people to use. Using a PC with adequate RAM and GPU capacity is also important to ensure larger models can run.
- Truncation length — as disclosed in *Results from Different Models*, scraped data truncation length must be varied depending on the size of model used. For a more indepth explanation, refer to the README in the github repository.
- Keywords — PDF\_KEYWORDS is a list of words that crawl4AI uses to prioritize PDFs and pages to crawl using the filter. It has been specified to certain keywords that commonly appear on pages with rebates and incentives. Feel free to add or subtract words to this list based on your expertise and findings.